
Training-Free Inference Acceleration for Pythia-70M: GQA, KVPress, and Layer-wise KV Compression

Anonymous Author(s)
Affiliation
Address
email

Abstract

1 Autoregressive language-model inference is limited by attention cost and
2 by the key-value (KV) cache, which grows linearly with context length.
3 This group project implements and evaluates several training-free accel-
4 eration strategies for Pythia-70M: a baseline SDPA runner, a reduced-KV
5 grouped-query attention (GQA) conversion, KVPress-Knorm, KVPress-
6 StreamingLLM, and an additional layer-wise sink-plus-recent KV com-
7 pression prototype. We evaluate WikiText-103 and one PG-19 test sample
8 using chunked perplexity, cached continuation perplexity, TTFT, TPOT,
9 throughput, and approximate KV-cache memory. The main finding is delib-
10 erately conservative: KV cache compression can improve CPU throughput
11 in selected settings, while naive training-free GQA reduces memory but
12 severely damages quality. The code is available at [https://github.com/](https://github.com/lilfry09/Training-free-LM-inference-acceleration-group/tree/layerwise-compression)
13 [lilfry09/Training-free-LM-inference-acceleration-group/tree/](https://github.com/lilfry09/Training-free-LM-inference-acceleration-group/tree/layerwise-compression)
14 [layerwise-compression](https://github.com/lilfry09/Training-free-LM-inference-acceleration-group/tree/layerwise-compression).

15 1 Introduction

16 The KV cache is essential for fast autoregressive decoding because each new token can
17 reuse keys and values from previous tokens. However, this cache also creates a memory
18 and attention bottleneck: for a model with N layers, context length L , KV heads H_{kv} , and
19 head dimension d , the cache is proportional to $2NLH_{kv}d$. The course assignment asks for
20 training-free acceleration on Pythia-70M, so the central question is not whether we can train
21 a better architecture, but which drop-in modifications remain honest and reproducible.

22 We integrate methods from two families. First, reduced-KV GQA stores fewer cached
23 key/value heads, inspired by uptrained GQA models (Ainslie et al., 2023). Second,
24 KV-cache compression removes token positions from the sequence dimension, following
25 the StreamingLLM attention-sink idea (Xiao et al., 2023) and the KVPress implementa-
26 tion (NVIDIA, 2024). We also implement a simple layer-wise schedule that keeps more
27 cache in shallow layers, compresses middle layers aggressively, and keeps a moderate budget
28 in deep layers.

29 Our contributions are: (1) a reproducible Pythia-70M evaluation framework for baseline,
30 GQA, KVPress-Knorm, KVPress-StreamingLLM, and layer-wise compression; (2) quality,
31 speed, and memory measurements on WikiText and PG-19; and (3) negative-result analysis
32 showing that training-free reduced-KV GQA is not a reliable accelerator for a pretrained
33 MHA checkpoint.

2 Methods

Baseline. The baseline loads the local EleutherAI Pythia-70M checkpoint through Hugging Face Transformers and uses scaled dot-product attention. Model parameters are never changed.

Reduced-KV GQA. Pythia-70M has 8 attention heads. Our GQA variant computes the original query, key, and value states, averages key/value heads into 2 cached KV heads, stores the reduced tensors in the dynamic cache, and evaluates grouped attention with PyTorch SDPA. This is a training-free conversion, not the uptraining recipe of Ainslie et al. (2023); therefore we treat it as a stress test and include a KV head sanity check.

KVPress compression. We evaluate KVPress-Knorm and KVPress-StreamingLLM at removal ratio 0.5. Knorm keeps cache entries with larger key-norm scores. StreamingLLM keeps sink tokens and recent tokens. Compression is applied after prefill through GPT-NeoX attention hooks, and the code records per-layer before/after cache lengths.

Layer-wise prototype. The additional prototype uses a deterministic sink-plus-recent selector with layer-specific keep ratios: shallow layers keep 80%, middle layers keep 50%, and deep layers keep 70%. It compresses once after prompt prefill and then continues greedy decoding with the compressed cache.

3 Experiments

Setup and metrics. All experiments use Pythia-70M, no training, CPU inference, and greedy decoding. WikiText uses the first 4096 tokens from the test split. PG-19 uses the first test example and takes the first 4096 Pythia tokens. Chunked PPL resets context every 512 tokens and is used for baseline/GQA quality. Cached PPL prefills 512 tokens, optionally compresses the cache, and scores the next 256 true tokens one by one. Speed uses 128, 512, and 1024 prompt tokens, one warm-up, three measured runs, and 64 generated tokens. We report TTFT, TPOT, throughput, and approximate KV memory. The main reproduction command is `.\scripts\run_matrix.ps1`.

Table 1: Quality on WikiText and PG-19. KVPress affects cached PPL, not chunked PPL, because compression is applied to generation caches.

Dataset	Method	Chunked PPL	Cached PPL	KV events	Removed
WikiText	Baseline	63.72	13.09	0	–
WikiText	GQA-2KV	2258.01	3659.00	0	–
WikiText	Knorm-0.5	63.72	52.44	78	0.50
WikiText	Stream-0.5	63.72	61.30	78	0.50
PG-19	Baseline	33.57	29.60	0	–
PG-19	GQA-2KV	816.70	776.88	0	–
PG-19	Knorm-0.5	33.57	31.76	78	0.50
PG-19	Stream-0.5	33.57	29.49	78	0.50

Table 1 shows the key quality result. GQA-2KV is a clear negative result: reducing 8 KV heads to 2 by averaging severely breaks the learned attention geometry. KVPress has a more useful trade-off. On PG-19, StreamingLLM-0.5 preserves cached PPL nearly exactly (29.60 to 29.49), while Knorm degrades it mildly. On WikiText, both KVPress variants hurt cached PPL, which shows that compression quality depends on the text slice and selector.

Table 2: Selected throughput results. Values are generated tokens/s; relative change compares the best compressed method to the baseline at the same context.

Dataset	Ctx	Baseline	Best compressed	Rel.	Method
WikiText	128	50.87	68.83	+35.3%	Stream
WikiText	512	34.95	52.54	+50.3%	Stream
WikiText	1024	51.24	43.89	-14.3%	Knorm
PG-19	128	74.31	66.41	-10.6%	Stream
PG-19	512	60.64	58.85	-3.0%	Stream
PG-19	1024	47.32	51.44	+8.7%	Stream

Table 2 summarizes speed rather than hiding the mixed outcome. KVPress-StreamingLLM improves WikiText throughput at 128 and 512 tokens, and PG-19 throughput at 1024 tokens, but not every setting improves. CPU overhead from hooks, indexing, and cache updates can dominate at some lengths; therefore we report raw TTFT/TPOT/throughput tables in `outputs/comparison_speed.md` and avoid claiming universal acceleration.

Table 3: Diagnostics on WikiText with a 512-token prompt. Cache MB is the approximate float32 per-sequence KV cache size.

Ablation	Setting	Cache MB	Chunked PPL	Cached PPL
GQA	8 KV heads	12.0	63.72	13.09
GQA	4 KV heads	6.0	1240.55	1976.45
GQA	2 KV heads	3.0	2258.01	3659.00
Stream	ratio 0.25	9.0	63.72	22.92
Stream	ratio 0.50	6.0	63.72	61.30
Stream	ratio 0.75	3.0	63.72	62.61

The diagnostics in Table 3 support two claims. First, the GQA path is numerically sane at 8 KV heads, then collapses as heads are reduced; the failure is the training-free conversion itself, not the grouped-attention implementation. Second, stronger StreamingLLM compression reduces memory but can worsen cached PPL. The layer-wise prototype further tests a heterogeneous budget: on a quick WikiText run it reduces cache by 28.4% and improves throughput from 16.89 to 36.41 tokens/s with PPL 80.36 to 82.30; on PG-19 it keeps the same cache reduction but slows down, again reinforcing that CPU speed depends on workload and overhead.

Group contribution statement. Current submitted workload: Fu Ruoyu implemented the method integration, reduced-KV GQA path, KVPress adapter, layer-wise compression prototype, experiment scripts, analysis, README, and report. Workload: 100%.

4 Conclusion

This study finds that not every training-free memory reduction is a valid accelerator. Naive reduced-KV GQA saves cache memory but breaks Pythia-70M quality. KVPress-style token compression is more promising and sometimes improves throughput, but its benefit is sensitive to dataset, context length, and implementation overhead. The best-supported conclusion is therefore an honest engineering one: KV cache compression is worth evaluating carefully under the target inference stack, while untrained architectural conversion should be treated as a negative baseline unless quality is repaired.

References

- Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebron, F., and Sanghai, S. GQA: Training generalized multi-query transformer models from multi-head checkpoints. EMNLP, 2023.
- Biderman, S., Schoelkopf, H., Anthony, Q., Bradley, H., O’Brien, K., Hallahan, E., and others. Pythia: A suite for analyzing large language models across training and scaling. ICML, 2023.
- Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. arXiv:1609.07843, 2016.
- NVIDIA. KVPress: KV cache compression for efficient LLM inference. <https://github.com/NVIDIA/kvpress>, 2024.

- 107 Rae, J. W., Potapenko, A., Jayakumar, S. M., and Lillicrap, T. P. Compressive transformers for
108 long-range sequence modelling. ICLR, 2020.
- 109 Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with
110 attention sinks. arXiv:2309.17453, 2023.